

API hooking에 대한 조사

정명훈

1. 정의

API Hooking = 프로그램이 원래 호출하려던 함수/API의 실행 흐름을 중간에서 가로채서, 내가 원하는 코드를 먼저 또는 대신 실행시키는 기술

2. 기본적인 사용 예시

프로그램이 원래 `IsDebuggerPresent();`를 호출한다고 생각하면 정상적으로는 프로그램 -> `IsDebuggerPresent()` -> Windows -> TRUE / FALSE 반환의 순서로 실행이 되어야 하지만 hooking을 걸면 프로그램 -> 내 Hook 함수 -> 원본 `IsDebuggerPresent()`과 같이 실행을 할 수 있다.

3. hooking이라고 불리는 이유

말 그대로 함수 호출에 갈고리(Hook)를 걸어서 중간에 낚아채기 때문에 hooking이라고 부른다. 예를 들어 원래 프로그램이 `int result = MessageBoxA(`

`NULL,`

`"Hello",`

`"Test",`

MB_OK

);를 실행한다고 생각하면 중간에 hook를 걸면 main.exe -> MyMessageBoxA() -> MessageBoxA()-> user32.dll 처럼 내가 만든 함수 MyMessageBoxA()를 먼저 실행시킬 수 있다. 여기서 MyMessageBoxA()함수에서

```
printf("MessageBoxA called!\n");
```

```
return OriginalMessageBoxA(
```

```
    hWnd,
```

```
    "HOOKED!",
```

```
    lpCaption,
```

```
    uType
```

); 위와 같은 작업을 수행하면 원래는 Hello가 출력되어야 하지만 실제로는 HOOKED!가 출력이 된다.

4. HOOKING을 왜 쓰는가?

Hooking 자체는 공격 기술이 아니고 정상 프로그램에서도 엄청 많이 쓰인다. 대표적으로 디버깅, API 호출 로깅, 모니터링, 호환성 계층, 샌드박스, 악성코드 분석에 쓰이며 특히 **리버싱**에서도 중요하다.

예를 들어서 어떤 프로그램이 파일을 CreateFileW(...)과 같은 api를 통해서 연다고 했을 때 이 api를 hook해서 다른 파일을 집어 넣으면 이 api가 정확히 무엇을 하려고 하는지를 알 수 있다.

5. Hooking은 어떻게 함수 호출을 가로채나?

이를 위해서 가장 중요하게 알아야 하는 것은 **프로그램이 함수의 주소를 어떻게 찾는가?** 이다.

windows프로그램은 dll함수를 사용할 때 대략 프로그램 -> IAT -> kernel32.dll -> CreateFileW의 순서로 함수의 주소를 접근하거나 아니면 call 00007FFB12345678과 같이 직접 주소로 접근하기도 한다. 그래서 hooking을 하려고 하는 사람의 입장에서는 **함수로 가는 주소나 코드 자체를 바꾸면 된다.** 따라서 다양한 api hooking방법이 존재하며

대표적으로IAT Hooking , Inline Hooking, EAT Hooking, VEH / Exception 기반 Hooking, Hardware breakpoint 기반 Hooking과 같은 방법이 있다.

6. IAT Hooking

이 중에서 iat hooking에 대해서 조금 더 자세히 알아보기 앞서서 iat란 (Import Address Table의 약자로 import된 함수들의 주소를 저장한 테이블이다. windows에서 프로그램을 실행하면 pe프로그램은 import table를 가지고 로더가 해당 ip주소를 찾아서 iat를 채워 넣는다.

예시) IAT CreateFileW→ 0x00007FFB12341000 , ReadFile→ 0x00007FFB12342000, MessageBoxA →0x00007FFB56781000

과 같이 말이다. 따라서 프로그램은 실제로 실행시 call qword ptr [__imp_MessageBoxA]코드를 __imp_MessageBoxA

-> 0x7FFB56781000 -> MessageBoxA로 메모리 관점에서 이동해서 실행을 할 수 있다. 그렇다면 iat hooking이란 간단

하게 iat 테이블에서 원래 IAT[MessageBoxA] -> MessageBoxA에서 IAT[MessageBoxA] -> MyHook로 바꾸는 것이다.

이러한 iat hooking의 장점은 원본 api코드를 건들지 않고 단순히 iat 테이블의 포인터만 바꿈으로서 구조가 단순하고 이해하기도 쉽다.

단점으로는 프로그램이 LoadLibrary(), GetProcAddress() 함수를 이용해 직접 api의 주소를 구하고 iat를 참조하지 않으면 통하지 않는다는 점이 있다. 예를 들어

```
HMODULE h = LoadLibraryA("user32.dll");
```

```
void *p = GetProcAddress(h, "MessageBoxA");, P(...., ...)과 같이 호출하면 IAT HOOKING을 쉽게 우회할 수 있다.
```

7. Inline Hooking

8. Inline Hooking은 함수 시작 부분의 실제 기계어 코드를 바꿔버리는 것으로서 예를 들면

MessageBoxA

+0x00 push rbx

+0x01 sub rsp,30

+0x05 mov rbx,rcx... 이 부분을 jmp MyHook와 같이 기계어 코드를 바꿔버린다.

이 방식을 통해서 Program -> MessageBoxA -> JMP MyHook -> Hook Code와 같이 자연스럽게 HOOK CODE로 이동을 하며 이는 X64DBG에서 함수 호출시 JMP를 통해 CODE CAVE로 이동하는 것과 그 흐름과 목적이 유사하지만 inline hook는 이를 일반화해서 **함수 호출 자체에 적용**한다는 차이점이 존재한다. 그 다음에 원래 함수를 호출하려고 하면 문제가 생기는데 함수의 시작을 jmp myhook로 적었기 때문에 **원래 함수를 호출하는 경우에 무한 루프에 빠진**다는 문제점이 발생한다.

이 문제점을 해결하기 위해서는 **Trampoline**이라는 것이 필요하다. 이는 쉽게 말하자면 앞에서 **jmp MyHook로 변경**시킨 MessageBoxA

```
+0x00 push rbx
```

```
+0x01 sub rsp,30
```

```
+0x05 mov rbx,rcx...
```

과 같은 부분의 명령어를 별도의 메모리로 복사한 다음

Trampoline:

```
+0x00 push rbx
```

```
+0x01 sub rsp,30
```

```
+0x05 mov rbx,rcx...
```

jmp A+원래뵤어쓰길과 같이 저장하고 원래 함수는 A: jmp MyHook + 원래A 코드....로 저장한다.

그리고 MyHook로 jmp한 뒤에 내가 할 일들을 하고 그 다음에 jmp Trampoline로 이동한다. 그렇게 하면 무한루프에 빠지지 않고 내 코드를 실행 후 원래 함수의 코드 역시 **정상적으로 실행할 수 있다.**

8.이러한 HOOK함수에서는 그럼 뭘 할 수 있는가?

굉장히 많은 걸 할 수 있다. 예를 들어 CreateFileW를 HOOK하면 wprintf(L"File opened: %s\n", lpFileName);와 같은 방식으로 파일 접근을 추적할 수 있다. SEND를 HOOK시 IP,PORT,전송 버퍼, 전송 패킷의 길이 등의 정보를 그리고 MALLOC를 HOOK하는 경우에는 메모리 할당을 추적할 수 있다.

9.리버싱에서 유용하게 사용하는 법에는 무엇이 있는가?

프로그램이 내부적으로 암호화 -> 복호화 -> Writefile을 한다고 치면 Writefile을 hook하면 **복호화 알고리즘을 완벽히 모르더라도 writefile을 통해서 최종 결과를 쉽게 알아낼 수 있다.** 따라서 hooking은 암호화,복호화 과정이 복잡할 때 정적 분석을 보완할 수 있는 매우 효과적이고 효율적인 방법이다.